# 人群圈选 Agent
# 让运营用一句话圈人，再证明它圈得准

这是个「说人话查数据」的小工具：运营在框里打一句「圈出近 30 天加购但没下单的高价值用户」， 系统自动翻译成 SQL、查出名单。但我真正花心思的不是把它做出来，而是搭一套有标准答案的评测，证明它到底准不准—— 最后把通过率从 **56.7%** 提到了 **96.7%**。

| 56.7% → 96.7% | 74.5% → 99.8% | 6 / 6 | 32 |
| --- | --- | --- | --- |
| 圈选通过率 | 平均 F1 | 兜底路由准确率 | 条分层评测用例 |

## 这份作品集讲什么

| 部分 | 想让你看到的 |
| --- | --- |
| 01 | 整体思路：先把"圈得对不对"的尺子立起来，再谈优化 |
| 02 | 背景：圈人这件事，卡在哪、大模型能帮上什么 |
| 03 | 问题怎么定的、优化点怎么一步步挖出来的 |
| 04 | 产品架构：三层结构 + 引擎里的四节点状态机 |
| 05–07 | 功能点、用户流程、原型界面 |
| 08 | 最终效果：三版 Prompt 的对照实验和数据 |

个人项目 · 2026.09 – 2026.10 · 独立完成 · Python / LangGraph / SQLite / LLM API

对标岗位：AI 产品运营 / 数据产品经理 / 增长产品经理

## 01 这项目到底在做什么

一句话：**先立起"圈得对不对"的尺子，再谈怎么优化。**

市面上「用自然语言查数据」的工具不少，但大多数只解决"能不能跑起来"，没几个认真回答"到底准不准"。 我一开始也是自己看着结果打分，写了个"准确率 88%"。后来一想，这数字站不住：**裁判是我自己，别人凭什么信?**

所以我干脆把整个项目的重心，从"做个 Agent"换成了"**搭一套能复现的评测**"。 做法是：给每条测试需求提前写好标准答案 SQL，让 Agent 跑出来的结果去和标准答案比，用集合重合度算分。 这样谁都能重跑一遍验证，分数是算出来的，不是拍脑袋拍出来的。

> 我在这项目里最看重的一次判断：自评准确率没说服力，能被人复现的评测才有。为此我宁可把 Agent 功能做得简单一点。

### 五步走下来

**1 先定尺子**
参考 SQL 跑出的标准人群包当答案，集合比对算 P/R/F1

**2 搭评测**
32 条分层用例，指标全自动，零人工打分

**3 拆开看**
按难度层拆通过率，直接看出短板在哪类需求

**4 改 Prompt**
3 版对照实验，每版只补上一版的洞

**5 包成产品**
把最优引擎封成一个能用的圈选工具

## 02 为什么做它

电商运营每天都在"圈人"：短信召回、优惠券定向、个性化推送，都得先从几千万用户里把目标人群挑出来， 比如"近 30 天加购没下单的高价值用户""流失预警的美妆用户"。

但圈人一直有两道坎：

| 痛点 | 具体表现 | 结果 |
| --- | --- | --- |
| **门槛高** | 要么自己会写 SQL，要么排队等 BI 出数 | 需求响应慢，运营卡在"提数"上 |
| **口径乱** | "活跃""老客""高潜"这些黑话，各人理解不一样 | 圈出来的不是同一批人，投放没法比 |

大模型能把一句话准确翻译成 SQL，让不会写代码的同事也能圈人。但新问题来了：**模型生成的结果对不对?** 如果没人能证明它的准确率，上线后每一次圈错，都是真金白银的投放浪费。

> 所以我给项目定了两层目标：① 做一个"一句话圈人"的工具；② 更关键的，搭一套评测把准确率量化出来，优化到可信。

# 03 问题怎么定的、优化点怎么挖的

## 先重新定义问题

最开始我图省事，自己看着结果打分。后来发现不对，把核心问题重新定义成一句话：

> **怎么在不靠人工打分的前提下，客观、可复现地度量"自然语言 → 人群包"这条链路的准确率，并据此找到短板？**

解法就是前面说的"标准答案当裁判"：

| 要素 | 我用的做法 |
| --- | --- |
| **标准答案** | 每条圈选需求预先写好参考 SQL，跑出"标准人群包" |
| **怎么比** | Agent 的结果和标准答案做集合重合度比对 |
| **算分指标** | Precision（查准）、Recall（查全）、F1（综合） |
| **通过线** | F1 ≥ 0.95 算通过，全程无人工打分 |

## 怎么找到该改的地方

只看一个总通过率，等于没信息。我把它拆成三层看：

| 层级 | 回答什么 | 它帮我发现了什么 |
| --- | --- | --- |
| **① 总通过率** | 整体好不好 | 基础版只有 56.7%，明显不能用 |
| **② 分层通过率** | 短板在哪类需求 | 矛盾层 0%、歧义层 33.3%，是硬伤 |
| **③ Bad case 归因** | 为什么会错 | 锁定四类错误，直接指导改 Prompt |

> **最值钱的是第 ② 层。**只有把通过率按难度拆开，才发现不同版本短板完全不一样：基础版在"矛盾需求"上是 0%（它把 AND 擅自改成了 OR），加了角色和示例之后矛盾层解决了，但"歧义口径"还卡在 33.3%。不拆层，我根本不知道该往哪改。

## 四类典型错误，对应四类改法

| 归因类型 | 典型需求 | 错在哪 | 怎么改 |
| --- | --- | --- | --- |
| **条件遗漏** | 近30天加购未下单的**高价值**用户 | 漏了 is_high_value | 用 Few-Shot 补组合场景 |
| **口径偏差** | "活跃用户" | 猜成"高价值用户" | 注入业务口径词典 |
| **时间口径错** | 近 7 天活跃 | 窗口天数 / 基准日算错 | 显式规定基准日锚点 |
| **矛盾处理错** | 年龄<18 且>60 | AND 改成 OR，人群膨胀 | 规则：矛盾按字面生成，空集即正确 |

# 04 架构长什么样

整体分三层，依赖方向是单向的：产品层 → 引擎层 → 数据层。引擎层完全不感知产品层，所以评测引擎能被产品层 零改动复用，也能单独跑回归。

**三层架构**



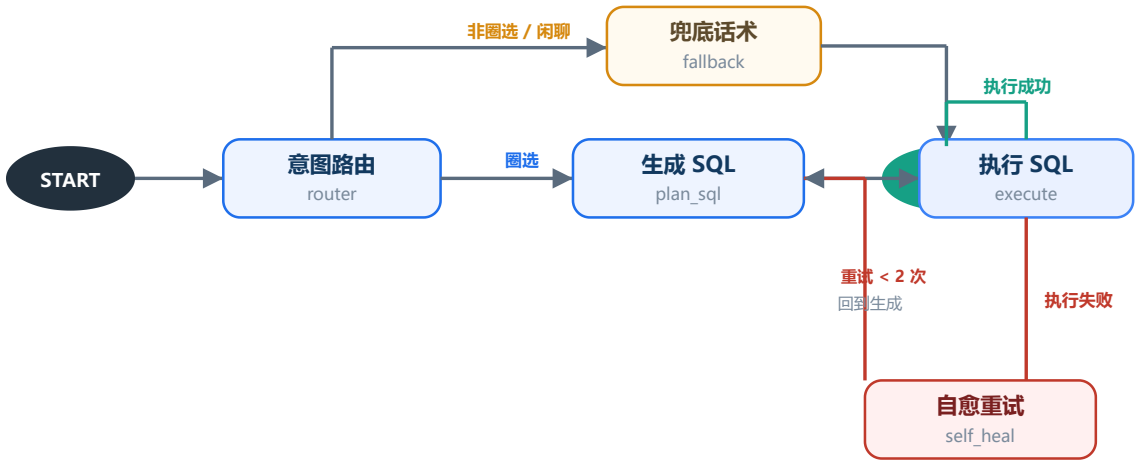

## 三个技术取舍（面试常问）

| 决策 | 选了 | 为什么 |
| --- | --- | --- |
| **Router 前置，不用 ReAct** | 先路由、后生成 | 圈选是模式化任务，入口先判意图（圈选还是闲聊）更可控、幻觉少 |
| **表结构注入，不上 RAG** | 全文塞进 Prompt | 单表 17 字段才几千 token；上千张表才需要向量检索，硬上反而多一个失败源 |
| **自愈重试带上限** | 最多 2 次 | 报错回传给模型修正很有用，但不设上限可能"越改越错"死循环 |

### 引擎内部：四节点状态机

把"需求 → 人群包"拆成四个职责单一的节点，自愈循环用条件边表达，而不是写死 if/else，链路可回溯、可观测。

**LangGraph 图结构（含自愈循环）**




## 05 有哪些功能

**自然语言圈选** —— 一句话进去，自动走完"路由 → 生成 SQL → 执行 → 自愈"，出来一份名单

**意图路由兜底** —— 识别圈选和闲聊，闲聊直接给话术，不硬生成 SQL，防幻觉

**SQL 透明可核对** —— 默认折叠，运营不关心、分析师要看，折叠兼顾两类人

**自愈重试** —— 执行报错自动把错误喂给模型修正，实测能修字段拼写错误

**人群画像** —— 圈完立刻看到性别 / 城市 / 会员 / 品类分布，先验一下圈得对不对

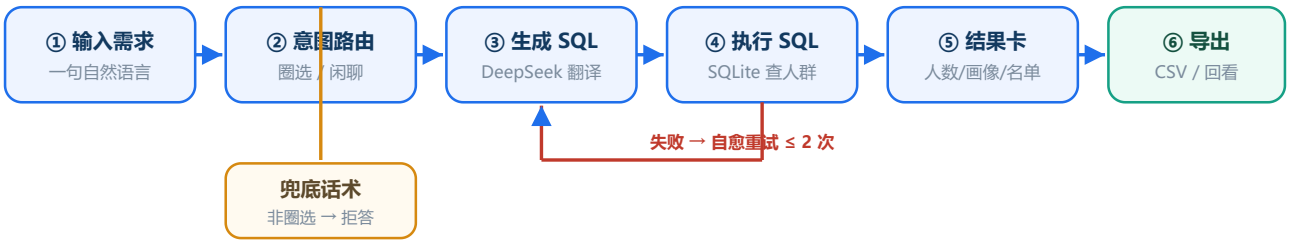
**名单导出 CSV** —— 一键导出完整名单（带 BOM，Excel 直开不乱码）

**圈选历史** —— 自动留痕，点一下回看完整结果，方便复盘复跑

# 06 用户怎么走完一圈

主流程六步：输入需求 → 意图路由 → 生成 SQL → 执行 → 结果卡 → 导出 / 回看。中间两个分支都做了兜底：非圈选 → 兜底话术；执行失败 → 自愈重试 ≤2 次。

**主流程与异常分支**




## 一次圈选背后的数据流转

```
POST /api/select
  → history.save() 占位拿 record_id
  → graph_flow.answer() 跑 LangGraph（路由 → 生成SQL → 执行 → 自愈≤2）
  → profile.stats() 出人群画像    → history.update() 回填结果
  → 组装 JSON → 前端渲染结果卡
```

业务库和历史库分开（audience.db / history.db），业务库重建时历史不受影响；引擎报错也不抛 500， 而是把失败原因当结果返回给用户——失败原因本身也是要展示的信息。

## 07 原型长啥样

做了个能直接跑的 Web 原型（FastAPI + 原生 JS，没构建步骤），界面走浅色业务工具风。下面是还原实际界面的示意：




### 几个设计上的小决定

| 设计点 | 为什么这么定 |
| --- | --- |
| **SQL 默认折叠** | 运营不关心 SQL，分析师要核对，折叠刚好兼顾两类人 |
| **画像用条形图** | 圈完第一反应是"这群人长什么样"，图比数字直观 |
| **名单只预览前 50 条** | 避免几千行卡死页面，完整名单走导出 |
| **结果内联在主区** | 方便连续圈选、对比前后结果 |
| **侧栏历史可回看** | 复盘和复跑方便，减少重复劳动 |

## 08 最后效果怎么样

把 Prompt 拆成三版做对照实验，每一版只加一类东西，这样收益能精确归因，而不是笼统一句"优化了"。

**56.7% → 96.7%**
通过率

**74.5% → 99.8%**
平均 F1

**6 / 6**
兜底路由

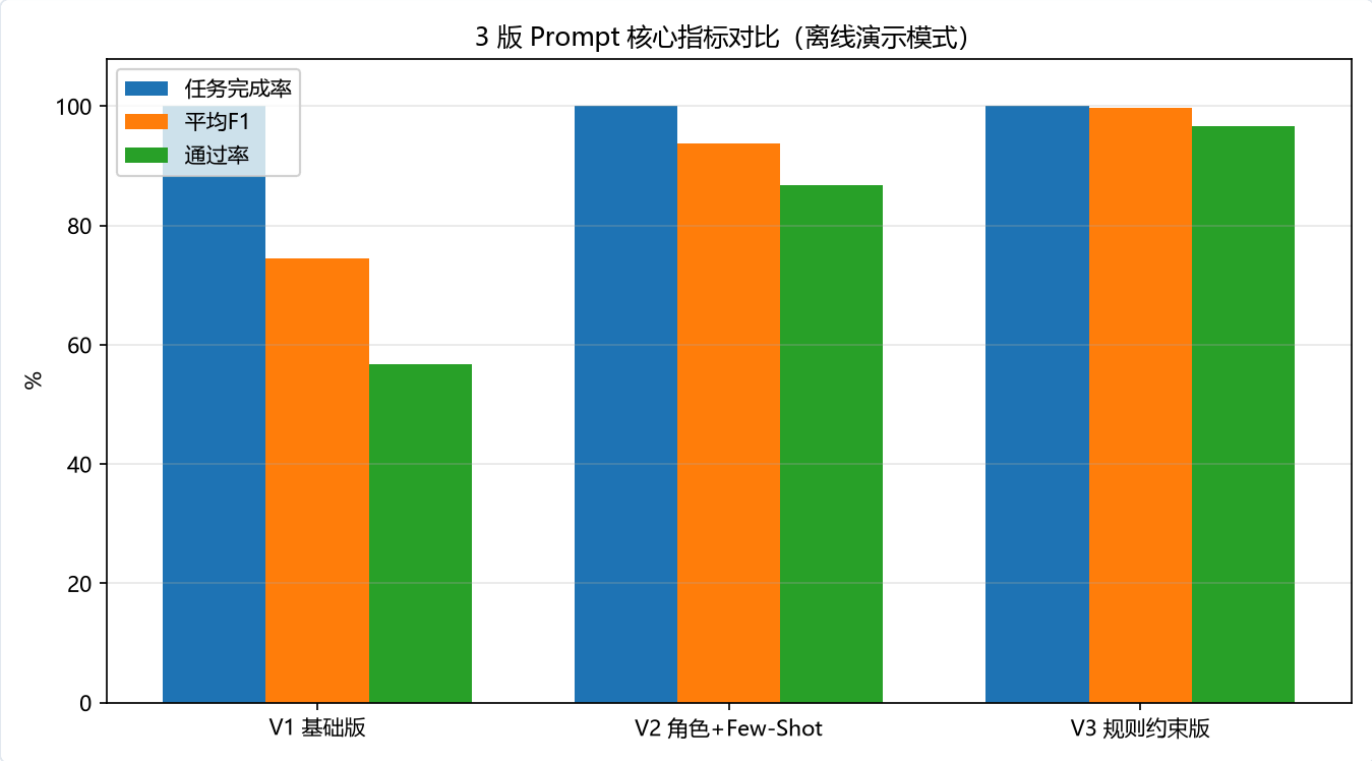



图 1　三版 Prompt 的任务完成率、平均 F1、通过率对比。

| 版本 | 加了什么 | 通过率 | 平均 F1 | 主要短板 |
|---|---|---|---|---|
| V1 基础版 | 只有表结构 | 56.7% | 74.5% | 矛盾层 0%、组合层 37.5% |
| V2 角色 + Few-Shot | 角色设定 + 2 个示例 + 口径词典 | 86.7% | 93.8% | 歧义层仍是 33.3% |
| V3 规则约束版 | 再加 6 条生成规则 | **96.7%** | **99.8%** | 仅时间层 83.3% |

每一版加的东西，都是上一版暴露的短板逼出来的。"角色 + Few-Shot"贡献主要提升，"规则约束"补齐了矛盾处理和口径边界。

## 分层通过率：拆开看，短板才露出来

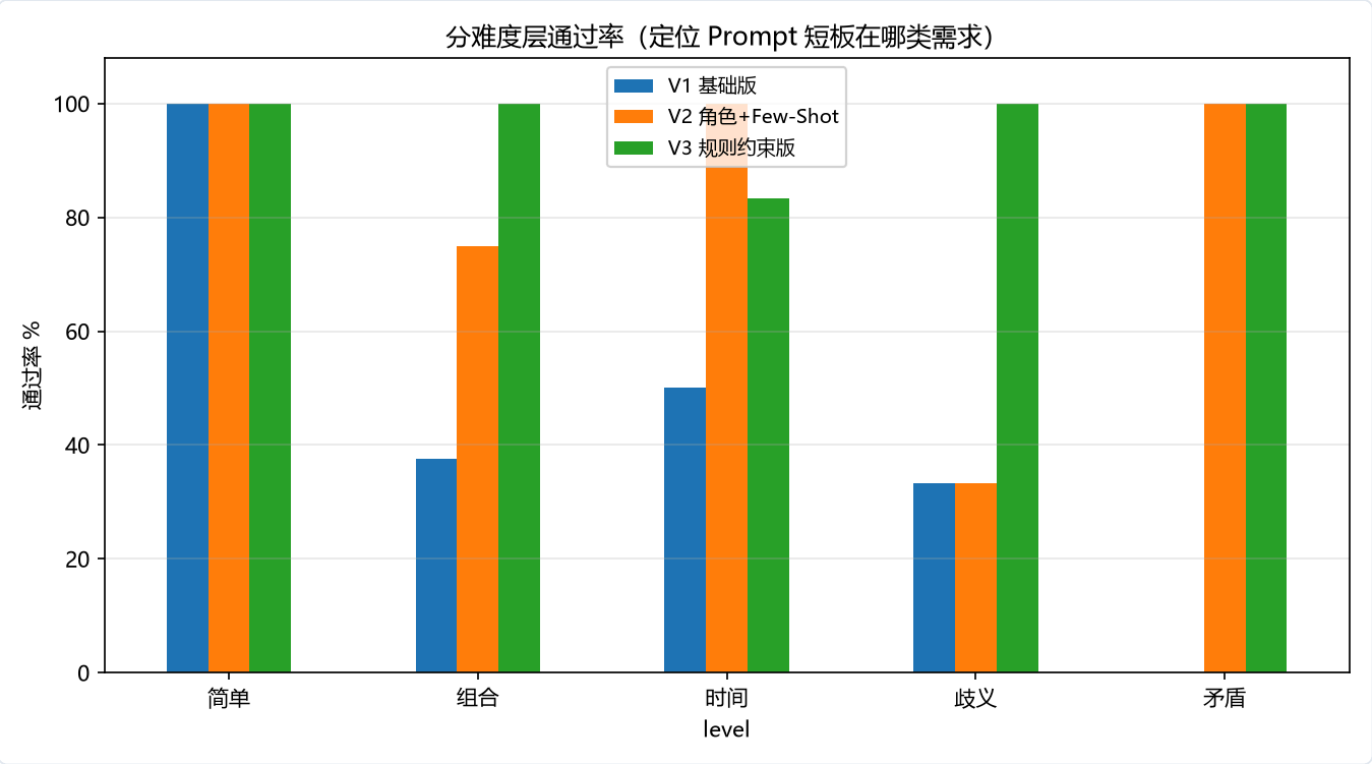



图 2　按难度层拆开的通过率。基础版在"矛盾"层是 0，"歧义"层 33.3%，一眼定位问题；只看总数看不出来。

| 难度层 | V1 基础版 | V2 角色+Few-Shot | V3 规则约束版 |
|---|---|---|---|
| 简单（10 条） | 100% | 100% | 100% |
| 组合（8 条） | 37.5% | 75% | **100%** |
| 时间窗口（6 条） | 50% | 100% | 83.3% |
| 歧义口径（3 条） | 33.3% | 33.3% | **100%** |
| 矛盾需求（3 条） | 0% | 100% | **100%** |

**关键洞察：**"歧义口径"（业务黑话）只靠词典注入没用，得配"必须查词典、不得自己猜"这条显式规则，才从 33.3% 收敛到 100%。 这说明"给模型的信息"和"给模型的约束"是两码事。

## Bad case 归因：四类错误全沉淀进模板

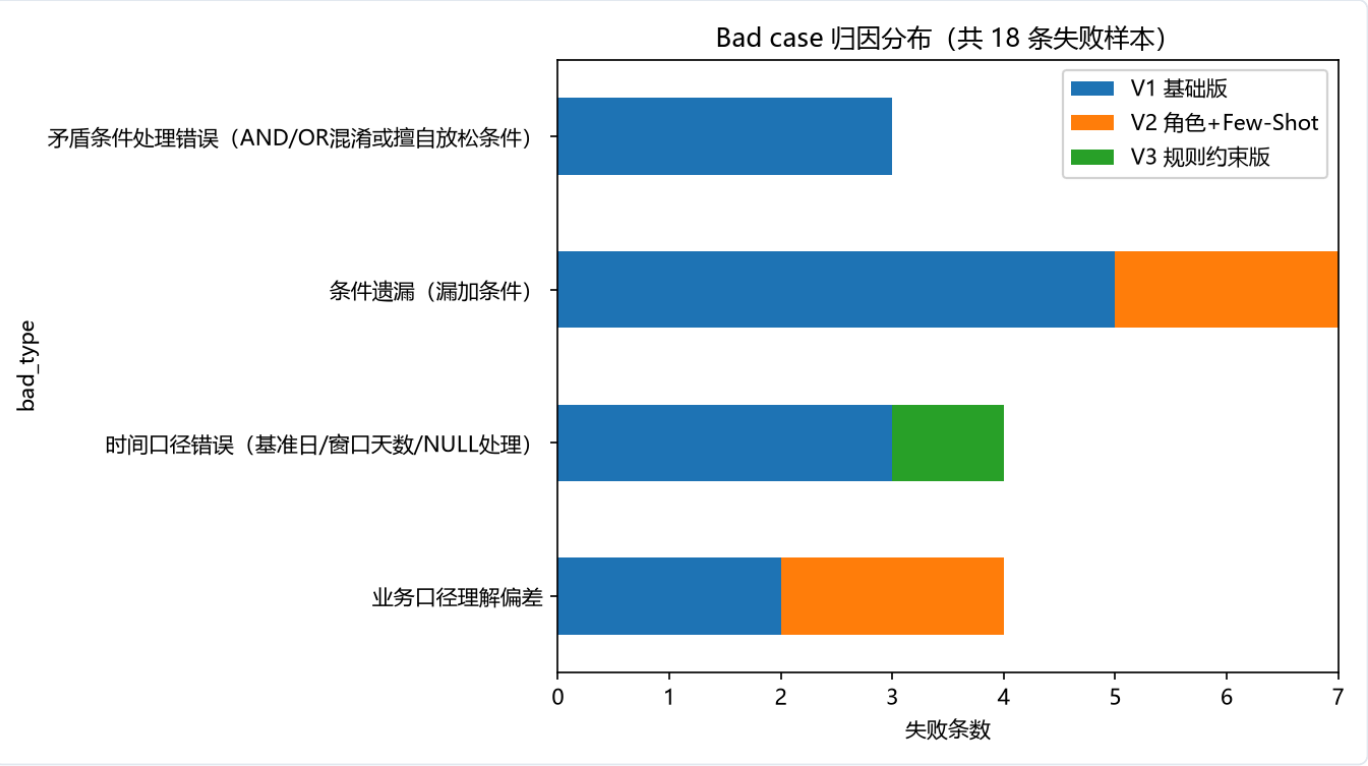



图 3　未通过用例的归因分布，四类错误对应四类 Prompt 优化动作。自愈也实测过：字段拼写错误被自动触发并修复，兜底路由 6/6。

## 复盘：做完这个项目，我最想说的三句话

**① 评测设计比功能实现值钱。**写 Agent 大概花了三成时间，剩下七成都在想"怎么证明它对"。 做完我更能理解为什么大模型团队都在死磕评测集——没有评测，优化全靠感觉，改十版也不知道是进步还是退步。

**② "准确率"必须能拆开。**只报一个总数字没信息量。我现在的做法是三层看：总通过率看整体、分层通过率看短板、 Bad case 归因看原因。三层里，分层性价比最高——它直接把"该改什么"指出来了。

**③ 模型的失败常常不是报错，是"看着合理其实错了"。**比如把 AND 擅自改成 OR、把"活跃"猜成"高价值"， 这些 SQL 都能执行成功，结果却全错。最隐蔽的错误往往最值钱。

## 如果继续做，下一步是什么

| 方向 | 具体做什么 |
| --- | --- |
| **换真实模型重跑** | 接 DeepSeek Key 把离线演示的数字换成真实结果 |
| **扩评测集** | 从 32 条扩到 100+，尤其补歧义、矛盾两层，96.7% 的置信度还不够高 |
| **加人工复核** | 机器判错的地方人工再过一遍，确认不是标准答案本身写错了 |

> 一句话收尾：这个项目的核心不是"做了一个圈选工具"，而是"建立了一套能证明它准不准、并且能一直把它改得更准的办法"。